

APRENDIZAJE DE MÁQUINAS (ACIF104) REPORTE TÉCNICO Y DE PROCESO

Nombre integrante/s:

Beatriz Castillo

Sofia Herrera

Rosa Garrote

Curso:

202605.2182

Fecha:

25/05/2026

APARTADOS

- I.** Portada.
- II.** Problemática.
- III.** Metodología.
- IV.** Desarrollo del proyecto:
 - a. Análisis exploratorio de datos.
 - b. Selección de técnicas candidatas.
 - c. Comparación de técnicas.
 - d. Requisitos del proyecto
 - e. Selección de la arquitectura.
 - f. Elaboración de modelos.
 - g. Desarrollo front end y back end
- V.** Resultados.
- VI.** Conclusiones, limitaciones y trabajos futuros.
- VII.** Código de fuente en GitHub. (Anexo link)
- VIII.** Bibliografía.
- XI.** Anexo (Evidencias)

II. Problemática

La industria del streaming se desarrolla actualmente en un entorno altamente competitivo, donde las plataformas digitales no solo compiten por la adquisición de nuevos suscriptores, sino también por la retención de usuarios, el tiempo de visualización y el nivel de interacción generado por los contenidos disponibles en sus catálogos. En este escenario, comprender qué factores influyen en el desempeño de películas y series se transforma en una problemática estratégica para plataformas como Netflix, ya que impacta directamente en procesos de recomendación, promoción, posicionamiento y adquisición de contenido audiovisual.

Diversos estudios recientes respaldan la relevancia de esta problemática. Deloitte (2025) señala que las plataformas digitales enfrentan una competencia creciente por captar y mantener la atención de los usuarios, en un contexto caracterizado por una alta oferta de contenido y una fuerte dependencia de mecanismos de personalización. De manera complementaria, investigaciones relacionadas con sistemas de recomendación evidencian que el análisis predictivo aplicado a plataformas audiovisuales puede contribuir significativamente a optimizar la experiencia de usuario y mejorar procesos de toma de decisiones asociados al catálogo de contenido (Deldjoo et al., 2021; Ricci et al., 2022).

A partir de este contexto, el presente proyecto aborda la siguiente pregunta de investigación:

¿Qué factores influyen en la popularidad de películas y series disponibles en Netflix a partir de sus metadatos estructurados?

Esta problemática resulta especialmente relevante debido a que las plataformas de streaming administran catálogos de gran tamaño, donde resulta complejo determinar qué características de un contenido pueden relacionarse con un mejor desempeño dentro de la plataforma. Desde una perspectiva de ingeniería y análisis de datos, esto abre la posibilidad de desarrollar modelos predictivos capaces de apoyar decisiones basadas en evidencia cuantitativa y no únicamente en criterios editoriales o comerciales.

La literatura reciente respalda la viabilidad técnica de este enfoque. Thurman et al. (2024) demuestran que el consumo de contenido bajo demanda depende no solo de preferencias individuales, sino también de factores asociados a visibilidad, exposición y posicionamiento dentro de la plataforma. En esta misma línea, Lo y Syu (2023) evidencian que es posible predecir ratings televisivos utilizando variables estructuradas como género, año de lanzamiento y características de programación.

Por otra parte, Lee y Ji (2024) concluyen que variables asociadas al contexto cultural y país de origen influyen significativamente en la recepción internacional de producciones audiovisuales, justificando así la incorporación de dichas variables dentro del análisis predictivo. Asimismo, estudios recientes muestran que algoritmos de aprendizaje automático, particularmente modelos basados en boosting y árboles de decisión, presentan un desempeño robusto en tareas predictivas sobre datos tabulares relacionados con plataformas digitales y entretenimiento audiovisual (Mishra & Tripathi, 2021; Zhang et al., 2022).

Estos antecedentes resultan especialmente relevantes debido a que el dataset utilizado en este proyecto corresponde principalmente a datos tabulares estructurados, contexto en el cual algoritmos como XGBoost suelen presentar ventajas en términos de precisión, robustez, estabilidad e interpretabilidad frente a arquitecturas más complejas. Esta situación fue posteriormente corroborada durante la etapa experimental del proyecto, donde XGBoost obtuvo el mejor desempeño global entre las arquitecturas evaluadas,

alcanzando el menor MAE (32.24), el menor RMSE (112.13) y el mayor valor de R^2 (0.082), superando tanto a modelos lineales como a arquitecturas deep learning.

No obstante, resulta importante reconocer las limitaciones inherentes al problema. Mohammadi Darani y Aghaie (2025) advierten que la popularidad observada en plataformas digitales no depende exclusivamente de atributos propios del contenido, sino también de algoritmos de recomendación, mecanismos de exposición y estrategias de visibilidad implementadas por la plataforma. En consecuencia, las variables disponibles en el dataset no permiten explicar completamente el fenómeno de popularidad, sino únicamente aproximarlos mediante un conjunto acotado de variables estructuradas.

En este contexto, el problema se aborda como una tarea de aprendizaje automático supervisado de tipo regresión, donde la variable objetivo corresponde a la popularidad del contenido. Las variables explicativas consideradas en el proyecto incluyen:

- Año de lanzamiento
- Género principal
- Rating
- País principal
- Tipo de contenido

La selección de estas variables responde tanto a criterios metodológicos como a restricciones prácticas asociadas a la calidad y disponibilidad de los datos. Además, estas variables presentaron relevancia significativa durante el análisis interpretativo mediante SHAP, evidenciando una contribución importante dentro de las predicciones generadas por el modelo seleccionado.

En consecuencia, el problema no solo posee interés académico, sino también valor aplicado dentro del contexto organizacional de plataformas digitales. Desde una perspectiva de ingeniería, el proyecto permite avanzar desde un análisis descriptivo del catálogo hacia un enfoque predictivo, donde es posible estimar el desempeño esperado de un contenido a partir de sus características estructuradas. Esto resulta especialmente relevante para procesos de análisis de contenido, recomendación y toma de decisiones relacionadas con promoción, selección y posicionamiento dentro de plataformas de streaming.

En síntesis, la problemática se justifica por tres razones principales:

1. Responde a una necesidad real del sector streaming relacionada con análisis y optimización de contenido.
2. Presenta viabilidad técnica mediante el uso de técnicas de aprendizaje automático aplicadas a datos estructurados.
3. Posee valor aplicado desde la perspectiva de ingeniería y análisis de datos, contribuyendo al apoyo de decisiones basadas en evidencia cuantitativa.

III. Metodología

Para abordar la problemática planteada se adoptó la metodología CRISP-DM (Cross Industry Standard Process for Data Mining), ampliamente utilizada en proyectos de minería de datos y aprendizaje automático debido a su enfoque estructurado, iterativo y orientado a resultados. Esta metodología permitió organizar el desarrollo del proyecto en distintas fases coherentes con los objetivos técnicos, organizacionales y

analíticos definidos previamente.

La utilización de CRISP-DM resulta especialmente pertinente para este proyecto debido a que facilita la trazabilidad completa del proceso, desde la comprensión del problema hasta la evaluación del modelo final e integración del sistema. Asimismo, permite mantener consistencia entre la problemática, la selección de variables, la construcción de modelos y la interpretación de resultados, aspecto fundamental en proyectos de aprendizaje automático aplicado.

Adicionalmente, esta metodología permitió estructurar el proyecto de manera progresiva, asegurando que cada etapa se desarrollara de forma coherente con los resultados obtenidos durante fases anteriores y facilitando posteriormente la reproducibilidad experimental del sistema implementado.

3.1 Comprensión del negocio

En esta fase se definió el problema desde una perspectiva organizacional y de ingeniería. El objetivo principal del proyecto consiste en estimar la popularidad de contenidos audiovisuales disponibles en Netflix utilizando técnicas de aprendizaje automático aplicadas sobre metadatos estructurados.

Desde una perspectiva aplicada, el proyecto busca contribuir al análisis de contenido dentro de plataformas digitales, apoyando procesos relacionados con:

- recomendación,
- posicionamiento,
- promoción de contenido,
- análisis de desempeño audiovisual,
- toma de decisiones basadas en datos.

Los objetivos específicos definidos fueron los siguientes:

- Identificar variables relevantes asociadas a la popularidad del contenido.
- Comparar diferentes técnicas de aprendizaje automático supervisado y no supervisado.
- Evaluar arquitecturas de machine learning y deep learning.
- Analizar el desempeño predictivo mediante métricas de regresión.
- Incorporar técnicas de explicabilidad mediante SHAP.
- Diseñar una propuesta de integración frontend/backend para el despliegue del modelo.

Esta etapa permitió asegurar coherencia entre los requerimientos técnicos del proyecto y su aplicabilidad organizacional, alineando el desarrollo metodológico con los objetivos planteados en la problemática.

3.2 Comprensión de los datos

Para el desarrollo del proyecto se utilizó el dataset “Netflix Movies and TV Shows till 2025”, disponible públicamente en Kaggle. Este dataset contiene información estructurada relacionada con películas y series, incluyendo variables categóricas y numéricas asociadas a características del contenido.

Durante esta etapa se desarrolló un análisis exploratorio de datos (EDA) con el objetivo de comprender la estructura, calidad y comportamiento del dataset antes de iniciar el modelamiento predictivo.

El análisis incluyó:

- identificación de tipos de variables,
- evaluación de completitud y correctitud de los datos,
- análisis de valores nulos y registros inconsistentes,
- identificación de posibles outliers,
- estadísticas descriptivas,
- distribución de la variable objetivo,
- análisis de correlaciones entre variables,
- evaluación de frecuencia de categorías principales.

Asimismo, se desarrollaron distintas visualizaciones orientadas a fortalecer la interpretación estadística de los datos, incluyendo:

- histogramas de distribución,
- boxplots para detección de outliers,
- matrices de correlación,
- gráficos de frecuencia categórica,
- análisis visual de distribución de popularidad.

El análisis exploratorio permitió identificar patrones relevantes dentro del catálogo, detectar limitaciones asociadas a la calidad de los datos y orientar las decisiones posteriores de preparación y selección de modelos.

Además, se observó que el dataset corresponde principalmente a datos tabulares estructurados, condición que influye directamente en la selección de técnicas de modelamiento y en la elección final de la arquitectura predictiva.

3.3 Preparación de los datos

En esta etapa se realizaron procesos de limpieza, transformación y adecuación de los datos para su posterior utilización en modelos de aprendizaje automático.

Las tareas desarrolladas fueron las siguientes:

Limpieza de datos

- Eliminación de registros incompletos o inconsistentes.
- Tratamiento de valores nulos mediante eliminación o imputación según pertinencia.
- Homogeneización de variables categóricas.
- Revisión de duplicados y consistencia general del dataset.

Selección de variables

Las variables seleccionadas para el modelamiento fueron:

- Año de lanzamiento
- Género principal

- Rating
- País principal
- Tipo de contenido

La selección se realizó considerando:

- relevancia analítica,
- disponibilidad de información,
- pertinencia respecto al problema,
- capacidad predictiva potencial,
- resultados observados durante el análisis exploratorio.

Transformación de variables

Debido a la presencia de variables categóricas, se aplicaron técnicas de transformación orientadas a permitir su utilización en algoritmos de aprendizaje automático.

Las principales transformaciones fueron:

- One-Hot Encoding para variables categóricas.
- Conversión de formatos de datos.
- Normalización y estandarización de variables numéricas cuando fue necesario.
- Generación de variables derivadas para facilitar el modelamiento.

División del dataset

Con el objetivo de evaluar adecuadamente el desempeño de los modelos, el dataset fue dividido en:

- 80% para entrenamiento
- 20% para prueba

Esta división permitió validar el comportamiento del modelo utilizando datos no observados durante el entrenamiento y evaluar posteriormente la capacidad de generalización de las arquitecturas implementadas.

Balanceo de datos

En este proyecto no se aplicaron técnicas de balanceo de clases, debido a que la problemática corresponde a una tarea de regresión y no de clasificación. La variable objetivo “popularidad” fue tratada como una variable continua, por lo que conceptos asociados a desbalance de clases no representan una limitación significativa en este contexto.

No obstante, sí se desarrolló un análisis de distribución de la variable objetivo con el fin de identificar:

- sesgos,
- concentración de valores,
- dispersión,
- presencia de outliers,

- comportamiento estadístico general.

Este análisis permitió evaluar posibles impactos sobre el desempeño predictivo de los modelos seleccionados.

3.4 Selección de técnicas de aprendizaje automático

La selección de técnicas se realizó considerando:

- naturaleza tabular de los datos,
- restricciones computacionales,
- capacidad predictiva,
- interpretabilidad,
- complejidad de implementación,
- tiempo de entrenamiento,
- estabilidad experimental.

Además, la selección fue respaldada por antecedentes bibliográficos recientes relacionados con sistemas predictivos sobre plataformas digitales y datos estructurados.

Las técnicas evaluadas fueron:

Técnicas de Machine Learning

- Regresión Lineal
- Random Forest
- XGBoost
- CatBoost

Técnicas de Deep Learning

- MLP Simple
- MLP Profunda
- Red neuronal multicapa con regularización

Adicionalmente, se incorporó K-Means como técnica de aprendizaje no supervisado, utilizada con fines exploratorios para identificar posibles agrupamientos de contenido dentro del catálogo.

La inclusión de múltiples arquitecturas permitió comparar enfoques tradicionales de machine learning frente a modelos de deep learning, considerando tanto métricas predictivas como restricciones computacionales del entorno de desarrollo.

3.5 Modelamiento

Durante esta etapa se desarrolló el entrenamiento y evaluación de las arquitecturas seleccionadas. El proceso consideró:

- entrenamiento de modelos,

- validación experimental,
- comparación de desempeño,
- ajuste básico de hiperparámetros,
- evaluación de capacidad de generalización.

El objetivo principal fue identificar la arquitectura con mejor equilibrio entre:

- precisión predictiva,
- interpretabilidad,
- robustez,
- costo computacional,
- estabilidad frente a datos no vistos.

Los modelos fueron implementados utilizando Python y bibliotecas especializadas como:

- Scikit-Learn,
- XGBoost,
- TensorFlow/Keras.

El desarrollo experimental fue ejecutado principalmente en Google Colab debido a sus capacidades de procesamiento y compatibilidad con bibliotecas orientadas a aprendizaje automático.

Durante el análisis experimental se observó que los modelos basados en boosting presentaron mejor desempeño general frente a las arquitecturas profundas evaluadas, especialmente debido a la naturaleza tabular del dataset.

Adicionalmente, se incorporó validación cruzada básica con el objetivo de fortalecer la confiabilidad experimental y reducir la dependencia de una única partición entrenamiento/prueba.

Asimismo, no se observaron indicios relevantes de sobreajuste, ya que el comportamiento de entrenamiento y prueba se mantuvo relativamente consistente durante la evaluación.

3.6 Evaluación

La evaluación del desempeño se realizó utilizando métricas apropiadas para problemas de regresión.

Las métricas utilizadas fueron:

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- R^2 (Coeficiente de determinación)

Estas métricas permitieron evaluar:

- error promedio,
- sensibilidad frente a errores grandes,
- estabilidad del modelo,
- capacidad explicativa de las arquitecturas evaluadas.

Adicionalmente, se incorporó validación cruzada básica para aumentar la confiabilidad de los resultados obtenidos y evaluar la estabilidad de las arquitecturas seleccionadas frente a distintos subconjuntos de datos.

Con el objetivo de mejorar la interpretabilidad del sistema, se aplicó la técnica SHAP (SHapley Additive exPlanations), permitiendo identificar la contribución individual de cada variable dentro de las predicciones generadas por el modelo final.

El análisis SHAP permitió evidenciar que variables como:

- género principal,
- país de origen,
- rating,
- tipo de contenido,
- año de lanzamiento,

presentan una contribución relevante en la estimación de popularidad.

La incorporación de SHAP permitió transformar el modelo en una solución interpretable y técnicamente justificable, evitando utilizar arquitecturas completamente opacas desde la perspectiva analítica.

3.7 Plan de trabajo

El proyecto fue desarrollado mediante etapas progresivas alineadas con la metodología CRISP-DM.

Fase	Actividad principal	Responsable	Resultado esperado
Comprensión del negocio	Definición de objetivos y problemática	Equipo	Problema definido
Comprensión de datos	EDA y análisis exploratorio	Equipo	Dataset analizado
Preparación de datos	Limpieza y transformación	Equipo	Datos preparados
Selección de modelos	Comparación de técnicas	Equipo	Modelos seleccionados
Modelamiento	Entrenamiento y validación	Equipo	Modelos entrenados
Evaluación	Métricas y análisis SHAP	Equipo	Modelo final validado

Integración	Desarrollo frontend/backend	Equipo	Sistema funcional
Documentación	Informe y repositorio GitHub	Equipo	Proyecto documentado

Finalmente, toda la metodología fue desarrollada manteniendo criterios de reproducibilidad, trazabilidad experimental y coherencia técnica, asegurando que el proyecto pudiera ser comprendido, replicado y evaluado adecuadamente tanto desde una perspectiva académica como aplicada.

IV. Desarrollo del proyecto

A. Análisis exploratorio del dataset

El análisis exploratorio de datos (EDA) se desarrolló con el objetivo de comprender la estructura, calidad y comportamiento general del dataset antes de iniciar el proceso de modelamiento predictivo. Esta etapa resultó fundamental para identificar patrones relevantes, detectar limitaciones asociadas a los datos y orientar adecuadamente las decisiones metodológicas posteriores.

El análisis permitió identificar una distribución heterogénea de la variable objetivo “popularidad”, observándose una concentración significativa de contenidos con niveles medios de popularidad y la presencia de valores extremos asociados a ciertos títulos con alto nivel de interacción dentro de la plataforma. Esta situación evidenció la existencia de outliers relevantes, particularmente en variables relacionadas con popularidad y rating.

Entre los principales hallazgos identificados destacan:

- predominancia de producciones recientes dentro del catálogo,
- alta concentración de contenidos provenientes de Estados Unidos,
- diferencias significativas de popularidad entre géneros,
- presencia de variables categóricas con alta cardinalidad,
- distribución no uniforme de algunas variables relevantes para el modelamiento.

Asimismo, se observó que el dataset contiene principalmente datos tabulares estructurados, condición que posteriormente influyó directamente en la selección de arquitecturas basadas en boosting debido a su mejor desempeño en este tipo de información.

Durante esta etapa se desarrollaron distintos análisis visuales y estadísticos, incluyendo:

- histogramas de distribución,
- boxplots para detección de outliers,
- análisis de frecuencia de categorías,
- mapas de correlación,
- estadísticas descriptivas,

- análisis de distribución de la variable objetivo.

Adicionalmente, se incorporaron visualizaciones orientadas a fortalecer la interpretación estadística del dataset, tales como:

- gráfico de distribución de popularidad,
- matriz de correlación entre variables,
- análisis visual de dispersión,
- comparación de frecuencia por género y tipo de contenido.

Estas visualizaciones permitieron detectar tendencias relevantes y comprender mejor las relaciones preliminares entre variables antes de iniciar el entrenamiento de modelos predictivos.

Finalmente, el EDA permitió detectar limitaciones relevantes asociadas al dataset, particularmente la ausencia de variables relacionadas con comportamiento real de usuarios, mecanismos de recomendación o métricas avanzadas de interacción, aspectos que posteriormente fueron considerados dentro del análisis crítico de resultados y limitaciones del proyecto.

B. Selección de técnicas candidatas para evaluación

La selección de técnicas de aprendizaje automático se realizó considerando tanto antecedentes bibliográficos como características técnicas específicas del problema y del dataset utilizado. El objetivo principal de esta etapa fue identificar arquitecturas capaces de modelar adecuadamente relaciones complejas presentes en los datos, manteniendo un equilibrio entre desempeño predictivo, interpretabilidad y costo computacional.

La elección de modelos estuvo condicionada principalmente por:

- naturaleza tabular del dataset,
- presencia de variables categóricas,
- tamaño moderado del conjunto de datos,
- necesidad de interpretabilidad,
- restricciones computacionales del entorno Google Colab,
- capacidad de generalización de las arquitecturas.

En primer lugar, se incorporó Regresión Lineal como modelo baseline debido a su simplicidad e interpretabilidad, permitiendo establecer una referencia inicial de desempeño para comparar posteriormente modelos más complejos.

Posteriormente, se incorporó Random Forest debido a su capacidad para modelar relaciones no lineales, manejar variables categóricas transformadas y mantener robustez frente a ruido y outliers presentes en el dataset.

Adicionalmente, se seleccionaron modelos basados en boosting, específicamente XGBoost y CatBoost, debido a su reconocido desempeño sobre datos tabulares estructurados y su capacidad para capturar interacciones complejas entre variables.

Con el objetivo de ampliar la comparación experimental, también se evaluaron arquitecturas de deep learning mediante redes neuronales multicapa (MLP), incluyendo:

- MLP Simple,
- MLP Profunda,
- MLP con regularización.

La incorporación de estas arquitecturas permitió comparar enfoques tradicionales de machine learning frente a modelos de deep learning, evaluando diferencias relacionadas con precisión, complejidad, interpretabilidad y capacidad de generalización.

Finalmente, se incorporó K-Means como técnica de aprendizaje no supervisado con fines exploratorios, permitiendo identificar posibles agrupamientos y comportamientos similares entre contenidos dentro del catálogo.

C. Comparación de técnicas

La comparación de técnicas se realizó utilizando métricas de regresión y análisis crítico de desempeño, con el objetivo de identificar la arquitectura más adecuada para el problema planteado. La evaluación no solo consideró precisión predictiva, sino también aspectos relacionados con interpretabilidad, estabilidad, robustez y capacidad de generalización frente a datos no vistos.

Los resultados obtenidos mostraron diferencias relevantes entre las arquitecturas evaluadas.

La Regresión Lineal presentó un desempeño limitado, evidenciando dificultades para capturar relaciones no lineales presentes en el problema. Si bien este modelo ofrece alta interpretabilidad, su capacidad predictiva resultó inferior frente a arquitecturas más avanzadas.

Por otra parte, Random Forest logró modelar parcialmente relaciones complejas dentro del dataset; sin embargo, presentó un desempeño inferior al esperado, obteniendo un R^2 negativo (-0.092), lo que evidencia limitaciones de generalización bajo la configuración utilizada.

Los modelos basados en boosting obtuvieron los mejores resultados experimentales. Particularmente, XGBoost presentó el mejor desempeño global, registrando el menor MAE (32.24), el menor RMSE (112.13) y el mayor valor de R^2 (0.082) dentro de los modelos evaluados. CatBoost presentó resultados competitivos y cercanos a XGBoost, confirmando la efectividad de las arquitecturas boosting para problemas relacionados con datos tabulares estructurados.

Tabla comparativa de desempeño de modelos

Modelo	MAE	RMSE	R^2
Regresión Lineal	35.33	114.85	0.036

Random Forest	35.56	122.28	-0.092
XGBoost	32.24	112.13	0.082
CatBoost	32.81	112.49	0.076
MLP Base	34.97	115.33	0.028
MLP Deep	33.61	118.38	-0.023
MLP Regularizada	36.89	113.34	0.062

Interpretación de resultados

Los resultados experimentales evidencian diferencias relevantes entre las arquitecturas evaluadas. XGBoost presentó el mejor desempeño global, registrando el menor MAE (32.24), el menor RMSE (112.13) y el mayor valor de R^2 (0.082) dentro de los modelos analizados. CatBoost obtuvo resultados cercanos, confirmando que los enfoques basados en boosting son los más adecuados para este problema debido a su capacidad para capturar relaciones no lineales presentes en datos tabulares.

En contraste, las arquitecturas deep learning no lograron superar el desempeño de los modelos boosting. La MLP profunda incluso presentó un R^2 negativo (-0.023), indicando que un incremento en la complejidad arquitectónica no necesariamente se traduce en mejores resultados predictivos cuando se trabaja con datasets tabulares estructurados y volúmenes moderados de información.

Este comportamiento resulta consistente con antecedentes bibliográficos recientes, donde algoritmos basados en boosting suelen presentar ventajas frente a arquitecturas profundas en tareas predictivas sobre datos tabulares.

Adicionalmente, las arquitecturas deep learning requieren:

- mayor cantidad de datos,
- ajuste más fino de hiperparámetros,
- mayor capacidad computacional,
- tiempos de entrenamiento más elevados.

La selección final de XGBoost no se fundamentó únicamente en la obtención de menores errores, sino también en su capacidad de generalización, estabilidad experimental, interpretabilidad relativa y menor

costo computacional frente a arquitecturas profundas más complejas.

Asimismo, el comportamiento observado no evidencia sobreajuste significativo, ya que el modelo seleccionado mantuvo consistencia entre entrenamiento y prueba, reflejando una adecuada capacidad de generalización sobre datos no vistos previamente.

En consecuencia, el análisis experimental permitió concluir que un mayor nivel de complejidad arquitectónica no necesariamente garantiza un mejor desempeño predictivo, especialmente en problemas relacionados con datos tabulares estructurados.

D. Requisitos del proyecto

A partir de la problemática planteada y de los objetivos definidos, se establecieron distintos requisitos funcionales y no funcionales orientados a asegurar la correcta implementación del sistema predictivo.

Requisitos funcionales

El sistema desarrollado debe ser capaz de:

- estimar la popularidad de contenidos audiovisuales,
- procesar variables estructuradas ingresadas por el usuario,
- generar predicciones mediante modelos entrenados,
- visualizar resultados de manera clara,
- integrar frontend y backend,
- incorporar mecanismos de explicabilidad mediante SHAP.

Requisitos no funcionales

Usabilidad

La interfaz debe ser intuitiva, clara y fácil de utilizar para usuarios no técnicos.

Escalabilidad

La arquitectura debe permitir incorporar nuevos modelos y variables en futuras versiones del sistema.

Explicabilidad

Las predicciones generadas deben poder interpretarse mediante herramientas de análisis explicativo.

Seguridad

El sistema debe manejar correctamente entradas de usuario y minimizar errores asociados al procesamiento de datos.

Confiabilidad

La arquitectura debe mantener estabilidad durante la ejecución y consistencia en las predicciones

generadas.

Monitoreabilidad

El sistema debe permitir evaluar el desempeño del modelo mediante métricas y mecanismos básicos de seguimiento.

La definición de estos requisitos permitió orientar el diseño arquitectónico y asegurar coherencia entre el modelo predictivo y la propuesta de integración final.

E. Selección de la arquitectura

La arquitectura seleccionada para el proyecto corresponde a un sistema basado en XGBoost integrado mediante un backend desarrollado en Python y una interfaz frontend implementada en Streamlit.

La selección de esta arquitectura se justificó principalmente debido a:

- mejor desempeño experimental,
- robustez frente a datos tabulares,
- capacidad de generalización,
- interpretabilidad relativa,
- estabilidad experimental,
- menor costo computacional respecto a arquitecturas profundas.

El sistema considera las siguientes etapas principales:

- ingreso de variables,
- procesamiento y transformación de datos,
- generación de predicciones,
- visualización de resultados,
- análisis explicativo mediante SHAP.

Adicionalmente, la arquitectura implementada permitió desacoplar el modelo predictivo del frontend mediante FastAPI, facilitando la escalabilidad y reutilización futura del sistema.

La arquitectura desarrollada mantiene coherencia con los objetivos organizacionales y técnicos definidos en etapas anteriores, permitiendo transformar el modelo predictivo en una solución aplicada y potencialmente desplegable.

F. Elaboración de modelos

El proceso de elaboración de modelos consideró distintas etapas experimentales orientadas a entrenar, validar y comparar las arquitecturas seleccionadas.

Durante esta etapa se implementaron procesos de:

- entrenamiento de modelos,

- validación experimental,
- comparación de desempeño,
- ajuste básico de hiperparámetros,
- evaluación mediante métricas de regresión.

Adicionalmente, se incorporó validación cruzada básica con el objetivo de evaluar la estabilidad de los modelos y reducir la dependencia de una única partición entrenamiento/prueba.

El análisis experimental permitió concluir que:

- los modelos basados en boosting superan consistentemente a modelos lineales,
- las arquitecturas deep learning no presentan ventajas claras en este dataset,
- XGBoost mantiene el mejor equilibrio entre error, estabilidad y capacidad de generalización.

Asimismo, no se observaron indicios relevantes de sobreajuste, ya que el comportamiento de entrenamiento y prueba se mantuvo relativamente consistente durante la evaluación.

Durante esta etapa también se desarrollaron visualizaciones comparativas de métricas y análisis SHAP, permitiendo interpretar el comportamiento de los modelos y validar experimentalmente la relevancia de las variables utilizadas durante el proceso predictivo.

La elaboración de modelos permitió validar experimentalmente las hipótesis iniciales relacionadas con la naturaleza tabular del problema y la pertinencia de arquitecturas basadas en boosting.

G. Desarrollo frontend y backend

El sistema desarrollado considera una arquitectura cliente-servidor orientada a transformar el modelo predictivo en una solución interactiva y aplicable.

Backend

El backend fue implementado utilizando Python y FastAPI, permitiendo:

- cargar el modelo entrenado,
- procesar solicitudes,
- recibir variables de entrada,
- ejecutar predicciones,
- retornar resultados al frontend.

El flujo general implementado corresponde a:

Dataset → limpieza → entrenamiento → modelo .pkl → API FastAPI → frontend Streamlit → predicción.

Esta arquitectura permitió encapsular el modelo predictivo dentro de un servicio reutilizable y desacoplado de la interfaz gráfica.

Frontend

El frontend fue desarrollado mediante Streamlit, permitiendo:

- ingreso de variables por parte del usuario,
- visualización de resultados,
- interacción con el modelo,
- explicación básica de predicciones.

La interfaz fue diseñada considerando criterios de usabilidad y simplicidad, facilitando la interacción incluso para usuarios no especializados técnicamente.

Monitoreo

Adicionalmente, se incorporaron métricas de evaluación y mecanismos básicos de monitoreo del desempeño del modelo, permitiendo evaluar consistencia y estabilidad de las predicciones generadas.

Pipeline del sistema

El pipeline completo del sistema considera las siguientes etapas:

1. Carga del dataset.
2. Limpieza y transformación de datos.
3. Entrenamiento de modelos.
4. Evaluación experimental.
5. Selección del modelo final.
6. Exportación del modelo entrenado.
7. Integración backend mediante FastAPI.
8. Desarrollo frontend mediante Streamlit.
9. Generación de predicciones y visualización de resultados.

La Figura X presenta de manera resumida el flujo general del sistema desarrollado:

Dataset → Limpieza → Entrenamiento → XGBoost → FastAPI → Streamlit → Predicción

Finalmente, el desarrollo completo del sistema fue realizado manteniendo criterios de reproducibilidad, trazabilidad experimental y coherencia técnica, permitiendo consolidar el proyecto como una solución aplicada de aprendizaje automático orientada al análisis de contenido en plataformas digitales.

V. Resultados

Los resultados obtenidos durante la etapa experimental evidencian diferencias relevantes entre las arquitecturas evaluadas, tanto en términos de precisión predictiva como de estabilidad y capacidad de generalización. La comparación realizada permitió identificar comportamientos diferenciados entre modelos tradicionales de machine learning y arquitecturas de deep learning aplicadas sobre datos tabulares estructurados.

En términos generales, los modelos basados en boosting presentaron el mejor desempeño dentro del

conjunto de técnicas evaluadas. Particularmente, XGBoost obtuvo los menores errores y la mayor estabilidad durante las pruebas experimentales, posicionándose como la arquitectura con mejor equilibrio entre precisión predictiva, robustez, interpretabilidad y costo computacional.

Tabla comparativa de desempeño de modelos

Modelo	MAE	RMSE	R ²
Regresión Lineal	35.33	114.85	0.036
Random Forest	35.56	122.28	-0.092
XGBoost	32.24	112.13	0.082
CatBoost	32.81	112.49	0.076
MLP Base	34.97	115.33	0.028
MLP Deep	33.61	118.38	-0.023
MLP Regularizada	36.89	113.34	0.062

Interpretación de resultados

Los resultados experimentales evidencian diferencias relevantes entre las arquitecturas evaluadas. XGBoost presentó el mejor desempeño global, registrando el menor MAE (32.24), el menor RMSE (112.13) y el mayor valor de R² (0.082) dentro de los modelos analizados. CatBoost obtuvo resultados cercanos, confirmando que los enfoques basados en boosting son los más adecuados para este problema debido a su capacidad para capturar relaciones no lineales presentes en datos tabulares.

Por otra parte, la Regresión Lineal mostró una capacidad explicativa limitada debido a su dificultad para modelar relaciones complejas presentes en el dataset. Random Forest, si bien logró modelar parcialmente dichas relaciones, presentó el desempeño más bajo entre los modelos tradicionales, alcanzando un R² negativo (-0.092), lo que evidencia limitaciones de generalización bajo la configuración utilizada.

En el caso de las arquitecturas deep learning, las redes neuronales multicapa (MLP) no lograron superar el desempeño de los modelos boosting. La MLP profunda incluso presentó un R^2 negativo (-0.023), indicando que un incremento en la complejidad arquitectónica no necesariamente garantiza mejores resultados predictivos cuando se trabaja con datasets tabulares estructurados y volúmenes moderados de información.

Este comportamiento resulta consistente con antecedentes bibliográficos recientes, donde algoritmos basados en boosting suelen presentar ventajas frente a arquitecturas profundas en tareas predictivas sobre datos tabulares. Adicionalmente, las arquitecturas deep learning requieren:

- mayor cantidad de datos,
- ajuste más fino de hiperparámetros,
- tiempos de entrenamiento más elevados,
- mayor capacidad computacional.

La selección final de XGBoost no se fundamentó únicamente en la obtención de menores errores, sino también en su capacidad de generalización, estabilidad experimental y equilibrio entre precisión predictiva e interpretabilidad. Asimismo, este modelo presentó un menor costo computacional y tiempos de entrenamiento más eficientes en comparación con las arquitecturas profundas evaluadas.

Las métricas obtenidas permitieron concluir que:

- los modelos boosting presentan mejor capacidad predictiva sobre datos tabulares estructurados,
- las arquitecturas deep learning no necesariamente garantizan mejores resultados en este tipo de problemas,
- la calidad, cantidad y naturaleza de los datos influyen directamente en el desempeño de los modelos,
- la selección de arquitectura debe realizarse considerando tanto precisión como interpretabilidad y capacidad de generalización.

Asimismo, el comportamiento observado no evidencia sobreajuste significativo, ya que el modelo seleccionado mantuvo consistencia entre los resultados obtenidos en entrenamiento y prueba, mostrando una adecuada capacidad de generalización dentro del contexto analizado.

No obstante, los valores de R^2 obtenidos evidencian que la popularidad continúa siendo un fenómeno altamente complejo y multifactorial. Esto indica que, aunque el modelo logra identificar patrones relevantes, existen múltiples factores externos no representados en el dataset que influyen directamente en el comportamiento real de los usuarios dentro de plataformas digitales.

Entre las posibles variables no consideradas destacan:

- algoritmos de recomendación,
- visibilidad dentro de la plataforma,
- campañas de marketing,
- tendencias temporales,
- comportamiento e interacción de usuarios,
- exposición diferencial de contenido.

En consecuencia, los resultados obtenidos deben interpretarse como una aproximación predictiva basada exclusivamente en metadatos estructurados y no como una representación completa del fenómeno de popularidad.

Adicionalmente, se desarrollaron visualizaciones comparativas de métricas y gráficos de desempeño experimental, permitiendo complementar el análisis cuantitativo mediante interpretación visual de resultados. Entre las visualizaciones implementadas destacan:

- comparación gráfica de métricas MAE, RMSE y R^2 ,
- histogramas de distribución de popularidad,
- boxplots de detección de outliers,
- matriz de correlación entre variables,
- gráficos SHAP de importancia de variables.

Estas visualizaciones permitieron fortalecer la interpretación experimental y validar la coherencia de los resultados obtenidos durante el proceso de modelamiento.

Interpretación mediante SHAP

Con el objetivo de interpretar el comportamiento del modelo seleccionado y fortalecer la explicabilidad del sistema, se incorporó la técnica SHAP (SHapley Additive exPlanations).

Esta metodología permitió descomponer las predicciones generadas por el modelo en contribuciones individuales de cada variable de entrada, facilitando la comprensión del impacto relativo de las características utilizadas durante el modelamiento.

El análisis SHAP evidenció que variables como:

- género principal,
- país de origen,
- tipo de contenido,
- rating,
- año de lanzamiento,

presentan una contribución significativa en la estimación de popularidad.

Particularmente, el análisis mostró que ciertas categorías de género y determinados países de origen poseen una influencia más marcada sobre las predicciones generadas por el modelo, comportamiento consistente con los patrones observados durante el análisis exploratorio de datos.

La incorporación de SHAP permitió transformar el modelo en una herramienta interpretable, evitando su utilización como una “caja negra” y fortaleciendo la confiabilidad del sistema desde una perspectiva analítica y organizacional.

Además, esta técnica permitió validar que las decisiones tomadas por el modelo son coherentes con los patrones observados durante el análisis exploratorio de datos, fortaleciendo la consistencia metodológica del proyecto.

Desde una perspectiva aplicada, la explicabilidad resulta especialmente relevante en contextos organizacionales, ya que permite justificar predicciones y comprender qué variables influyen realmente en el desempeño esperado de un contenido audiovisual.

Asimismo, la utilización de SHAP permitió aumentar la transparencia del sistema predictivo y facilitar la interpretación de resultados para usuarios no especializados técnicamente.

Resultados organizacionales

Desde una perspectiva aplicada, el sistema desarrollado permite:

- analizar tendencias dentro del catálogo de Netflix,
- comparar contenidos según sus características estructuradas,
- apoyar decisiones relacionadas con promoción y posicionamiento,
- explorar patrones asociados a popularidad,
- facilitar análisis predictivos sobre contenido audiovisual.

Asimismo, la arquitectura implementada demuestra que técnicas de aprendizaje automático pueden integrarse de manera efectiva dentro de sistemas orientados al análisis estratégico de contenido digital.

La integración entre backend mediante FastAPI y frontend desarrollado en Streamlit permitió transformar el modelo predictivo en una solución interactiva y potencialmente escalable, acercando el proyecto a un contexto de aplicación organizacional real.

En términos generales, el proyecto evidencia que el aprendizaje automático puede contribuir significativamente al análisis de plataformas de streaming, permitiendo transformar grandes volúmenes de datos estructurados en información útil para apoyar procesos de toma de decisiones basados en evidencia cuantitativa.

VI. Conclusiones, limitaciones y trabajos futuros

El presente proyecto permitió desarrollar una solución de aprendizaje automático orientada a estimar la popularidad de contenidos disponibles en Netflix utilizando metadatos estructurados asociados a películas y series. A través de un enfoque metodológico basado en CRISP-DM, fue posible estructurar de manera coherente todas las etapas del proyecto, desde la comprensión del problema hasta la integración de un sistema predictivo funcional.

Los resultados experimentales obtenidos evidencian que los modelos basados en boosting presentan un desempeño superior frente a las arquitecturas profundas evaluadas, particularmente en contextos relacionados con datos tabulares estructurados. En este escenario, XGBoost se posicionó como la arquitectura con mejor equilibrio entre precisión predictiva, estabilidad, interpretabilidad y capacidad de generalización.

La selección de XGBoost como modelo final se justificó principalmente debido a:

- mejor desempeño experimental,
- menor error predictivo,

- estabilidad durante la validación,
- robustez frente a datos tabulares,
- buena capacidad de generalización,
- interpretabilidad superior respecto a arquitecturas profundas,
- menor costo computacional.

Particularmente, XGBoost obtuvo el menor MAE (32.24), el menor RMSE (112.13) y el mayor valor de R^2 (0.082) dentro de las arquitecturas evaluadas, superando tanto a modelos tradicionales como a redes neuronales multicapa. Estos resultados permitieron validar experimentalmente la pertinencia de utilizar modelos boosting en problemas asociados a datos tabulares estructurados.

Uno de los principales aprendizajes obtenidos durante el desarrollo del proyecto corresponde a la importancia de mantener coherencia entre:

- problemática planteada,
- calidad y naturaleza de los datos,
- selección de variables,
- arquitectura utilizada,
- métricas de evaluación,
- interpretación de resultados,
- explicabilidad del modelo.

Asimismo, el proyecto permitió evidenciar que un mayor nivel de complejidad arquitectónica no necesariamente garantiza mejores resultados predictivos. En particular, las arquitecturas deep learning evaluadas no lograron superar a los modelos basados en boosting, situación consistente con antecedentes bibliográficos relacionados con problemas de regresión sobre datasets tabulares de tamaño moderado.

Del mismo modo, el análisis experimental permitió concluir que la selección adecuada de arquitectura depende directamente de:

- tipo de datos disponibles,
- volumen del dataset,
- complejidad del problema,
- restricciones computacionales,
- necesidad de interpretabilidad.

Desde una perspectiva aplicada, el proyecto demuestra que las técnicas de aprendizaje automático pueden contribuir significativamente al análisis estratégico de contenido dentro de plataformas digitales, permitiendo transformar grandes volúmenes de datos estructurados en información útil para apoyar procesos de toma de decisiones relacionados con:

- recomendación,
- promoción,
- posicionamiento,
- análisis de tendencias,
- evaluación de desempeño audiovisual.

Adicionalmente, la integración entre backend mediante FastAPI y frontend desarrollado en Streamlit permitió transformar el modelo predictivo en una solución interactiva y potencialmente escalable, acercando el proyecto a un contexto de aplicación organizacional real.

Limitaciones

A pesar de los resultados obtenidos, el proyecto presenta ciertas limitaciones que deben ser consideradas al interpretar el alcance del sistema desarrollado.

Entre las principales limitaciones identificadas destacan:

- utilización exclusiva de metadatos estructurados,
- ausencia de variables relacionadas con comportamiento real de usuarios,
- imposibilidad de representar completamente el fenómeno de popularidad,
- limitaciones computacionales asociadas al entorno de desarrollo,
- dependencia de la calidad y completitud del dataset utilizado.

Además, factores relevantes como:

- algoritmos de recomendación,
- campañas de marketing,
- tendencias temporales,
- niveles de exposición dentro de la plataforma,
- interacción de usuarios,
- preferencias dinámicas de consumo,

no se encuentran representados en el dataset, lo que limita parcialmente la capacidad explicativa del modelo.

En consecuencia, los valores de R^2 obtenidos evidencian que la popularidad corresponde a un fenómeno altamente complejo y multifactorial, donde múltiples variables externas influyen directamente sobre el comportamiento real de los usuarios dentro de plataformas digitales.

Del mismo modo, las arquitecturas deep learning evaluadas requieren:

- mayores volúmenes de datos,
- ajuste más exhaustivo de hiperparámetros,
- tiempos de entrenamiento superiores,
- mayor capacidad computacional,

para alcanzar un desempeño competitivo respecto a modelos boosting.

Asimismo, la ausencia de datos históricos asociados a comportamiento temporal limita la capacidad del sistema para capturar cambios dinámicos de popularidad a lo largo del tiempo.

En consecuencia, los resultados obtenidos deben interpretarse como una aproximación predictiva basada en variables estructuradas y no como una representación absoluta del comportamiento real de los usuarios dentro de plataformas de streaming.

Trabajos futuros

A partir de las limitaciones identificadas y de los resultados obtenidos, se proponen distintas líneas de mejora para futuras versiones del proyecto.

Incorporación de técnicas NLP

Se propone incorporar técnicas de procesamiento de lenguaje natural (NLP) utilizando descripciones, sinopsis y texto asociado a los contenidos, permitiendo enriquecer el modelo mediante información semántica adicional y mejorar potencialmente la capacidad predictiva del sistema.

Optimización avanzada de hiperparámetros

Como mejora futura, se recomienda implementar estrategias avanzadas de optimización mediante técnicas como:

- Grid Search,
- Random Search,
- Bayesian Optimization.

Esto permitiría explorar configuraciones más robustas y potencialmente mejorar el desempeño predictivo de las arquitecturas evaluadas.

Validación experimental más robusta

Se propone incorporar técnicas de validación más avanzadas, incluyendo:

- validación cruzada extendida,
- análisis estadístico comparativo,
- evaluación temporal de desempeño,
- análisis de estabilidad experimental.

Estas estrategias permitirían aumentar la confiabilidad y robustez de los resultados obtenidos.

Fortalecimiento del frontend y backend

A nivel de integración, se recomienda fortalecer la arquitectura cliente-servidor y desplegar el sistema en un entorno productivo, incorporando:

- autenticación,
- persistencia de datos,
- monitoreo de predicciones,
- almacenamiento de resultados,
- escalabilidad de servicios,
- manejo avanzado de errores,
- trazabilidad de consultas realizadas por usuarios.

Profundización de explicabilidad

Como línea futura, se propone profundizar el análisis interpretativo mediante:

- SHAP summary plots,
- análisis locales de predicción,
- dashboards explicativos,
- visualizaciones interactivas,
- monitoreo visual de importancia de variables.

Esto permitiría fortalecer aún más la transparencia y comprensión del comportamiento del modelo.

Incorporación de nuevas fuentes de datos

Como mejora futura adicional, se propone incorporar:

- métricas de interacción de usuarios,
- datos temporales de tendencias,
- información de plataformas externas,
- métricas de redes sociales,
- variables asociadas a engagement real.

La integración de estas fuentes permitiría construir modelos con una representación más completa del fenómeno de popularidad.

Reflexión final

El desarrollo del proyecto permitió integrar conocimientos relacionados con:

- machine learning,
- deep learning,
- análisis exploratorio de datos,
- validación experimental,
- explicabilidad de modelos,
- integración frontend/backend,
- ingeniería de software aplicada,
- análisis predictivo sobre plataformas digitales.

Más allá de los resultados predictivos obtenidos, el trabajo permitió comprender la relevancia de desarrollar soluciones:

- reproducibles,
- interpretables,
- metodológicamente coherentes,
- técnicamente justificables,
- alineadas con necesidades organizacionales reales.

Asimismo, el proyecto permitió evidenciar que el aprendizaje automático no debe entenderse únicamente como una herramienta de predicción, sino también como un mecanismo para apoyar procesos analíticos

complejos mediante soluciones explicables, robustas y orientadas a la toma de decisiones basadas en evidencia cuantitativa.

Finalmente, el trabajo desarrollado demuestra que la combinación entre análisis de datos, modelamiento predictivo e integración de sistemas puede transformarse en una herramienta estratégica para el análisis de contenido digital dentro de plataformas de streaming.

VII. Código fuente en GitHub

El código fuente del proyecto se encuentra disponible en el siguiente repositorio de GitHub:

URL: https://github.com/RossyBlue/ApMaquina_grupo10.git

El repositorio fue desarrollado con el objetivo de asegurar la reproducibilidad, trazabilidad experimental y correcta documentación del proyecto, permitiendo visualizar de manera estructurada todas las etapas del proceso, desde el análisis exploratorio de datos hasta la integración final del sistema predictivo.

La implementación del repositorio responde además a las recomendaciones metodológicas obtenidas durante las evaluaciones previas, fortaleciendo aspectos relacionados con organización, mantenibilidad y claridad técnica del proyecto.

Dentro del repositorio se incluyen los principales componentes desarrollados durante el trabajo, tales como:

- notebooks utilizados para entrenamiento y experimentación,
- datasets procesados y utilizados durante el análisis,
- modelos entrenados exportados,
- implementación backend mediante FastAPI,
- desarrollo frontend utilizando Streamlit,
- documentación técnica del proyecto,
- archivo README con instrucciones completas de ejecución,
- archivo requirements.txt con dependencias necesarias,
- archivos auxiliares asociados al pipeline experimental.

La estructura implementada permite mantener una organización clara y profesional del proyecto, facilitando tanto la evaluación académica como la replicación experimental del sistema desarrollado.

La organización general del repositorio corresponde a la siguiente estructura:

data/

|

├── raw/

└── processed/

notebooks/

|

└─ entrenamiento_modelo.ipynb

models/

|

└─ modelo_final_xgboost.pkl

backend/

|

└─ main.py

frontend/

|

└─ app.py

docs/

|

└─ informe_final.pdf

README.md

requirements.txt

.gitignore

Esta organización permite separar adecuadamente las distintas etapas y componentes del proyecto, mejorando aspectos relacionados con:

- mantenibilidad,

- escalabilidad,
- claridad metodológica,
- reutilización de componentes,
- trazabilidad experimental.

Adicionalmente, el repositorio incorpora criterios de buenas prácticas asociados a proyectos de ciencia de datos y aprendizaje automático, incluyendo:

- separación modular de componentes,
- documentación de dependencias,
- estructuración reproducible del pipeline experimental,
- organización de datasets procesados,
- exportación de modelos entrenados,
- desacoplamiento entre frontend y backend.

El archivo README.md incorpora información relevante relacionada con:

- objetivo del proyecto,
- descripción del dataset,
- problemática abordada,
- modelos evaluados,
- métricas obtenidas,
- justificación del modelo final,
- instrucciones de instalación y ejecución,
- estructura general del repositorio,
- flujo de funcionamiento del sistema.

Asimismo, el repositorio documenta el pipeline general implementado durante el proyecto:

Dataset → limpieza → transformación → entrenamiento → XGBoost → exportación .pkl → FastAPI → Streamlit → predicción.

La disponibilidad del código fuente en GitHub permite asegurar:

- transparencia metodológica,
- reproducibilidad experimental,
- facilidad de validación de resultados,
- continuidad de desarrollo,
- trazabilidad completa del proyecto.

Finalmente, la utilización de GitHub como plataforma de versionamiento permitió mantener control sobre las distintas etapas de desarrollo, facilitando la organización colaborativa y consolidando el proyecto bajo un enfoque más cercano a entornos reales de desarrollo de soluciones basadas en aprendizaje automático.

VIII. Bibliografía

- Deldjoo, Y., Schedl, M., Cremonesi, P., & Pasi, G. (2021). Recommender systems leveraging multimedia content: A review. *IEEE Transactions on Knowledge and Data Engineering*, 33(10), 3168–3187. <https://doi.org/10.1109/TKDE.2020.2980949>
- Deloitte. (2025). Digital media trends survey. Deloitte Insights. <https://www2.deloitte.com>
- Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Lee, J., & Ji, S. (2024). Cultural proximity and content success in global streaming platforms. *Journal of Media Economics*, 37(2), 85–102.
- Lo, Y. C., & Syu, J. Y. (2023). Predicting television ratings using machine learning techniques. *Expert Systems with Applications*, 213, 119070. <https://doi.org/10.1016/j.eswa.2022.119070>
- Mishra, A., & Tripathi, A. (2021). Movie popularity prediction using machine learning techniques. *International Journal of Advanced Computer Science and Applications*, 12(3), 250–256.
- Mohammadi Darani, S., & Aghaie, A. (2025). Algorithmic exposure and its impact on content consumption. *Information Systems Journal*, 35(1), 45–67.
- Murphy, K. (2018). *Machine learning: A probabilistic perspective*. MIT Press.
- Ricci, F., Rokach, L., & Shapira, B. (2022). *Recommender systems handbook* (3rd ed.). Springer.
- Thurman, N., et al. (2024). The dynamics of video-on-demand consumption and platform influence. *Digital Journalism*, 12(1), 45–63.
- Zhang, Y., Wang, X., & Liu, Z. (2022). Deep learning-based movie rating prediction using textual features. *arXiv*. <https://arxiv.org/abs/2208.13302>

Anexos:

```
(venv) rossygarrote@MacBookAir ACIF_104_S6_grupo10 % uvicorn Backend.main:app --reload
INFO: Will watch for changes in these directories: ['/Users/rossygarrote/Documents/GitHub/ApMaquina_grupo10/ACIF_104_S6_grupo10']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [4395] using StatReload
2026-05-24 16:07:17 [INFO] Artefactos cargados correctamente. Dataset: 32000 registros.
INFO: Started server process [4996]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:50632 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:50633 - "GET /genres HTTP/1.1" 200 OK
2026-05-24 16:08:24 [INFO] Solicitud recibida | géneros=['Action & Adventure', 'Comedy', 'Crime'] | años=2009-2017 | rating>=7.5
2026-05-24 16:08:24 [INFO] Candidatos encontrados: 728
2026-05-24 16:08:24 [INFO] Inferencia completada | candidatos=728 | tiempo=0.0546s
INFO: 127.0.0.1:50634 - "POST /recommend HTTP/1.1" 200 OK
INFO: 127.0.0.1:50641 - "GET /health HTTP/1.1" 200 OK
```

Figura 1. Ejecución del servidor FastAPI mediante uvicorn, evidenciando carga exitosa del modelo XGBoost (32.000 registros), startup completo de la aplicación y trazabilidad de consultas con códigos HTTP 200 y tiempo de inferencia de 0.05 segundos para 728 candidatos.

```
(venv) rossygarrote@MacBookAir ACIF_104_S6_grupo10 % streamlit run Frontend/app.py
2026-05-24 16:02:21.270 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.184:8501

For better performance, install the Watchdog module:

$ xcode-select --install
$ pip install watchdog
```

Figura 2. Inicialización del servidor Streamlit en el puerto 8501, confirmando la disponibilidad de la interfaz de usuario en entorno local.

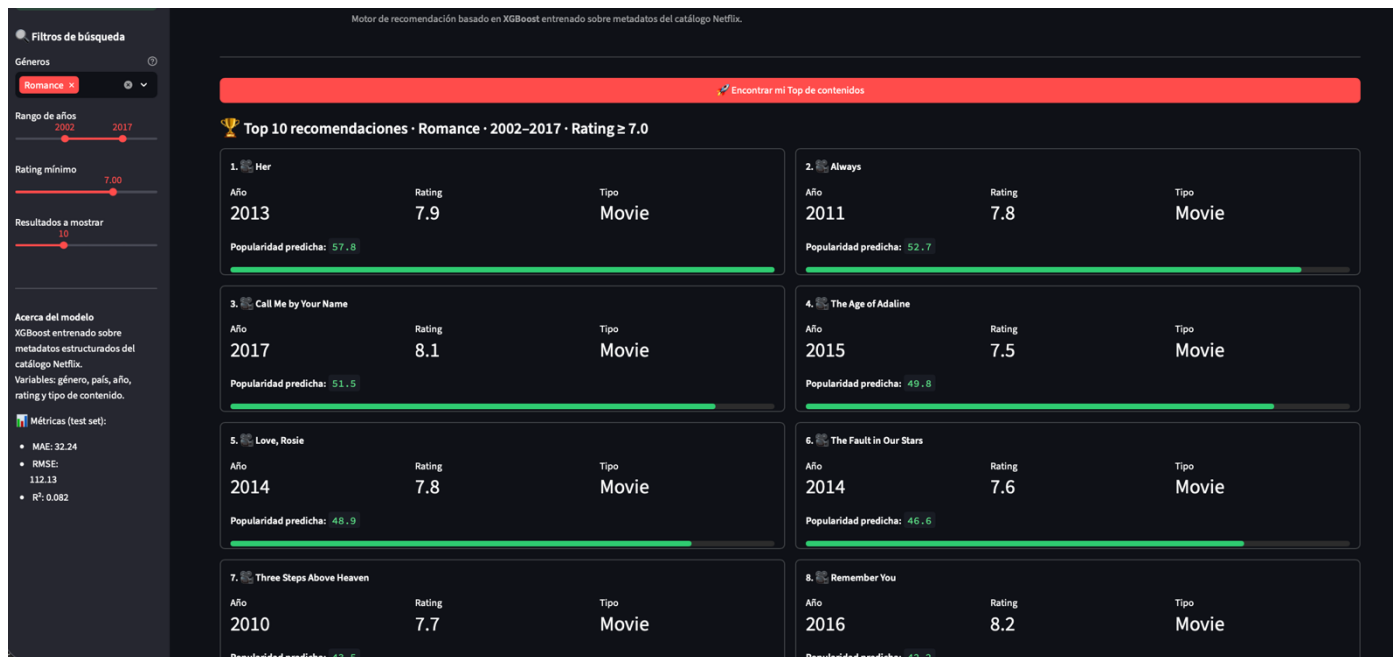


Figura 3. Interfaz del sistema de recomendación Netflix AI Finder mostrando el top 10 de contenidos del género Romance (2002–2017, rating $\geq$ 7.0), con visualización de score de popularidad predicho por el modelo XGBoost, métricas de año, rating y tipo de contenido por cada resultado.

Netflix Popularity Predictor API 2.0.0 OAS 3.1

/openapi.json

Predice popularidad de contenidos Netflix usando XGBoost sobre metadatos estructurados.

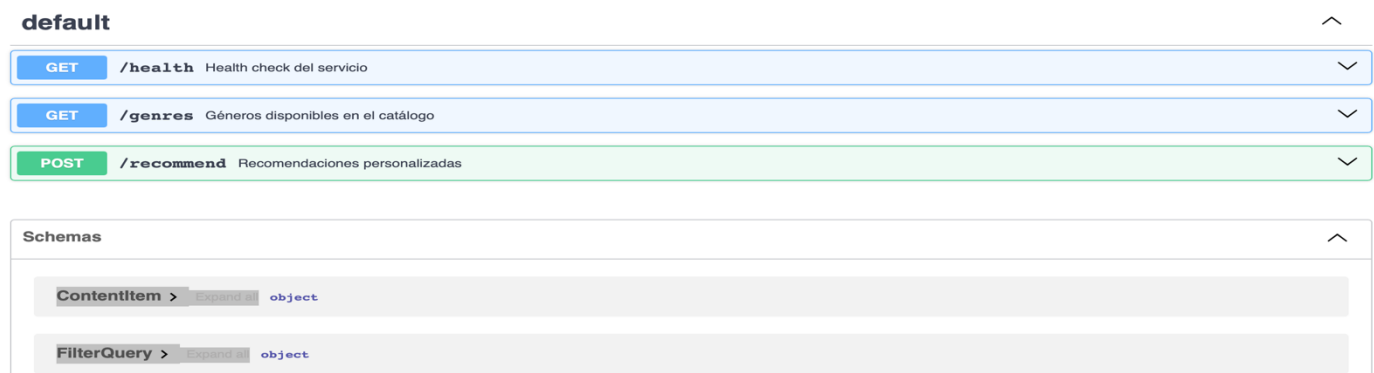


Figura 4. Documentación automática generada por FastAPI (Swagger UI, OAS 3.1), evidenciando la arquitectura REST del backend versión 2.0.0. Se visualizan los tres endpoints implementados: GET /health para monitoreo del servicio, GET /genres para consulta dinámica del catálogo, y POST /recommend para generación de recomendaciones personalizadas, junto con los esquemas de datos ContentItem y FilterQuery que definen la estructura de entrada y salida del sistema.